Guru Ghasidas Vishwavidyalaya

Session: 2026

Major:

Object Oriented Programming with Java

Assignment for the fulfillment of Lab Course

SUBMITTED TO

Dr. Ratnesh Shrivastava
Associate Professor

SUBMITTED BY

Priyadarshan Pradhan
BSc. (H) Computer Science
4th Semester
GGV/24/05153

Dept. of Computer Science & Information Technology

1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmer's name.

```java
import java.util.*;

import java.io.*;


class Farmer implements Comparable<Farmer> {

private String name;
private String fruit;
private String city;
private int quantity;


public Farmer(String name, String fruit, String city, int quantity) {
this.name = name;
this.fruit = fruit;
this.city = city;
this.quantity = quantity;
}

public String getName() { return name; }
public String getFruit() { return fruit; }
public String getCity() { return city; }
public int getQuantity(){ return quantity; }

@Override
public int compareTo(Farmer other) {
return Integer.compare(this.quantity, other.quantity);
}

public String toTableRow() {
return String.format("| %-18s | %-12s | %-15s | %8d |",
name, fruit, city, quantity);
}

@Override
public String toString() { return toTableRow(); }
}
public class Problem1_FarmerManager {

static final String HEADER =
"+——————————————————+——————————————+——————————————————+——————————+\n" +
```

```
"| Farmer Name | Fruit | City | Quantity |\n" +
"+-------------------+-------------+----------------+----------+";
static final String FOOTER =
"+-------------------+-------------+----------------+----------+";


static String buildTable(List<Farmer> list, String title) {
StringBuilder sb = new StringBuilder();
sb.append(title).append("\n");
sb.append(HEADER).append("\n");
for (Farmer f : list) {
sb.append(f.toTableRow()).append("\n");
}
sb.append(FOOTER).append("\n");
return sb.toString();
}


static void saveToFile(String filename, String content) throws IOException {

try (FileWriter fw = new FileWriter(filename)) {
fw.write(content);
}
System.out.println("[File saved] -> " + filename);
}


static void searchByName(List<Farmer> list, String query) {
System.out.println("\n-- Search results for \"" + query + "\" --");
boolean found = false;
for (Farmer f : list) {
if (f.getName().equalsIgnoreCase(query)) {
System.out.println(HEADER);
System.out.println(f.toTableRow());
System.out.println(FOOTER);
found = true;
}
}
if (!found) System.out.println("No farmer found with name: " + query);
}
public static void main(String[] args) throws IOException {
Scanner sc = new Scanner(System.in);
List<Farmer> farmers = new ArrayList<>();
System.out.println("╔═══════════════════════════════════╗");
System.out.println("║ Farmer Fruit Management System ║");
System.out.println("╚═══════════════════════════════════╝");
System.out.println("Enter details for 10 farmers:\n");
```

```java
for (int i = 1; i ≤ 10; i++) {
System.out.println("-- Farmer " + i + " --");
System.out.print(" Name : "); String name = sc.nextLine().trim();
System.out.print(" Fruit : "); String fruit = sc.nextLine().trim();
System.out.print(" City : "); String city = sc.nextLine().trim();
System.out.print(" Quantity : "); int qty = Integer.parseInt(sc.nextLine().trim());
farmers.add(new Farmer(name, fruit, city, qty));
System.out.println();
}

String beforeTable = buildTable(farmers, "=== BEFORE SORTING ===");
System.out.println("\n" + beforeTable);
saveToFile("farmers_before_sort.txt", beforeTable);
Collections.sort(farmers);
String afterTable = buildTable(farmers, "=== AFTER SORTING (by Quantity) ===");
System.out.println("\n" + afterTable);
saveToFile("farmers_after_sort.txt",
beforeTable + "\n\n" + afterTable);
System.out.print("\nEnter farmer name to search: ");
String query = sc.nextLine().trim();
searchByName(farmers, query);
sc.close();
}
}
```

OUTPUT:

```
╔═══════════════════════════════════════════════════════════╗
╔═══════════════════════════════════════════════════════════╗
╔═══════════════════════════════════════════════════════════╗
```

```
| Amitesh            | Grapes       | Nagpur          |      300 |
| Ravi               | Pineapple    | Bhopal          |      350 |
| Keshav             | Papaya       | Korba           |      270 |
| Joseph             | Jackfruit    | Kochi           |      430 |
| Chedil             | Banana       | Aizwal          |      530 |
+--------------------+--------------+-----------------+----------+


[File saved] -> farmers_before_sort.txt


=== AFTER SORTING (by Quantity) ===
+--------------------+--------------+-----------------+----------+
| Farmer Name        | Fruit        | City            | Quantity |
+--------------------+--------------+-----------------+----------+
| Raju               | Guava        | Kolkata         |      120 |
| Keshav             | Papaya       | Korba           |      270 |
| Amitesh            | Grapes       | Nagpur          |      300 |
| Abhishek           | Apple        | Jammu           |      320 |
| Ravi               | Pineapple    | Bhopal          |      350 |
| Joseph             | Jackfruit    | Kochi           |      430 |
| Venkatesh          | Mango        | Raipur          |      432 |
| Ranjan             | Mango        | Sambalpur       |      530 |
| Chedil             | Banana       | Aizwal          |      530 |
| Adil               | Lichi        | Muzzafarpur     |      540 |
+--------------------+--------------+-----------------+----------+


[File saved] -> farmers_after_sort.txt


Enter farmer name to search: Ranjan


── Search results for "Ranjan" ──
+--------------------+--------------+-----------------+----------+
| Farmer Name        | Fruit        | City            | Quantity |
+--------------------+--------------+-----------------+----------+
| Ranjan             | Mango        | Sambalpur       |      530 |
+--------------------+--------------+-----------------+----------+
```

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```java
import java.util.*;


enum Quality {
GOOD, BAD, AVERAGE;

public static Quality fromString(String s) {
```

```
return Quality.valueOf(s.trim().toUpperCase());
}
}

class FruitEntry {
private final String fruitName;
private final int quantity;
private final Quality quality;

public FruitEntry(String fruitName, int rawQuantity, Quality quality) {
this.fruitName = fruitName;
this.quantity = (rawQuantity > 1000) ? rawQuantity / 2 : rawQuantity;
this.quality = quality;
}

public String getFruitName() { return fruitName; }
public int getQuantity() { return quantity; }
public Quality getQuality() { return quality; }


@Override
public String toString() {
return String.format(" Fruit: %-8s | Quality: %-8s | Quantity: %d",
fruitName, quality, quantity);
}
}

public class Problem2_FruitInventory {

static final List<String> ALLOWED_FRUITS =
Arrays.asList("MANGO", "APPLE", "PAPAYA");
static Map<String, List<FruitEntry>> inventory = new HashMap<>();
static void addEntry(String fruit, int rawQty, Quality quality) {
String key = fruit.toUpperCase();
inventory.computeIfAbsent(key, k -> new ArrayList<>())
.add(new FruitEntry(fruit, rawQty, quality));
if (rawQty > 1000) {
System.out.println(" [Rule] Quantity " + rawQty
+ " > 1000 → stored as " + (rawQty / 2));
}
}
static void search(String fruitName, Quality targetQuality) {
System.out.println("\n── Search: " + fruitName + " | Quality: " + targetQuality + " ──");
String key = fruitName.toUpperCase();

if (!inventory.containsKey(key)) {
System.out.println(" No entries found for fruit: " + fruitName);
```

```java
return;
}

List<FruitEntry> entries = inventory.get(key);
boolean found = false;
for (FruitEntry e : entries) {
if (e.getQuality() == targetQuality) {
System.out.println(e);
found = true;
     }
}
if (!found) {
System.out.println(" No " + fruitName + " with quality '"
+ targetQuality + "' found.");
     }
}

static void printInventory() {
System.out.println("\n╔═══════════════════════════════╗");
System.out.println("║ Current Inventory ║");
System.out.println("╚═══════════════════════════════╝");
for (String key : inventory.keySet()) {
System.out.println(" [" + key + "]");
for (FruitEntry e : inventory.get(key)) {
System.out.println(" " + e);
            }
     }
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.println("╔═══════════════════════════════════╗");
System.out.println("║ Fruit Quality-Quantity System ║");
System.out.println("╚═══════════════════════════════════╝");
System.out.println("Fruits available: Mango, Apple, Papaya");
System.out.println("Quality options : good, bad, average\n");
for (String fruit : ALLOWED_FRUITS) {
System.out.println("-- " + fruit + " --");
int qty = -1;
while (qty < 0) {
System.out.print(" Quantity : ");
try { qty = Integer.parseInt(sc.nextLine().trim()); }
catch (NumberFormatException e) {
System.out.println(" Please enter a valid integer.");
     }
}
```

```
Quality q = null;
while (q == null) {
System.out.print(" Quality : ");
try { q = Quality.fromString(sc.nextLine()); }
catch (IllegalArgumentException e) {
System.out.println(" Invalid! Use: good / bad / average");
     }
}

addEntry(fruit, qty, q);
System.out.println();
}
printInventory();
System.out.println("\n— Search Section —");
System.out.print("Enter fruit name (Mango/Apple/Papaya): ");
String searchFruit = sc.nextLine().trim();
Quality searchQ = null;
while (searchQ == null) {
System.out.print("Enter quality to search (good/bad/average): ");
try { searchQ = Quality.fromString(sc.nextLine()); }
catch (IllegalArgumentException e) {
System.out.println("Invalid! Use: good / bad / average");
     }
}

search(searchFruit, searchQ);
sc.close();
     }
}
```

OUTPUT:

```
╔═══════════════════════════════════╗
║     Fruit Quality-Quantity System    ║
╚═══════════════════════════════════╝

Fruits available: Mango, Apple, Papaya

Quality options : good, bad, average


-- MANGO --

  Quantity : 1500

  Quality  : good
```

```
  [Rule] Quantity 1500 > 1000 → stored as 750

-- APPLE --
  Quantity : 400
  Quality  : bad

-- PAPAYA --
  Quantity : 1200
  Quality  : average
  [Rule] Quantity 1200 > 1000 → stored as 600
╔═══════════════════════════════════╗
║         Current Inventory         ║
╚═══════════════════════════════════╝

  [MANGO]
    Fruit: MANGO    | Quality: GOOD     | Quantity: 750  (was 1500, halved)

  [APPLE]
    Fruit: APPLE    | Quality: BAD      | Quantity: 400

  [PAPAYA]
    Fruit: PAPAYA   | Quality: AVERAGE  | Quantity: 600  (was 1200, halved)
── Search Section ──
Enter fruit name (Mango/Apple/Papaya): Mango
Enter quality to search (good/bad/average): good

── Search: Mango | Quality: GOOD ──
  Fruit: MANGO    | Quality: GOOD     | Quantity: 750
── Search Section ──
Enter fruit name (Mango/Apple/Papaya): Apple
Enter quality to search (good/bad/average): good
```

```
── Search: Apple | Quality: GOOD ──
  No Apple with quality 'GOOD' found.

- - - - - - - - - - - - - - - - - - -

Enter fruit name (Mango/Apple/Papaya): Papaya
Enter quality to search (good/bad/average): GOOOD
  Invalid! Use: good / bad / average
Enter quality to search (good/bad/average): average

── Search: Papaya | Quality: AVERAGE ──
  Fruit: PAPAYA   | Quality: AVERAGE  | Quantity: 600
```

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

The available orchards in a tree-view.

Search for a given fruit in the available orchards after taking input from the user & print the tree.

```java
import java.util.*;

interface Searchable {
boolean matchesFruit(String fruitName);
boolean matchesQuality(String quality);
}

abstract class Orchard implements Searchable {
private final String orchardName;
private final String fruitName;
private final String quality;

protected Orchard(String orchardName, String fruitName, String quality) {
this.orchardName = orchardName;
this.fruitName = fruitName;
```

```java
this.quality = quality;
}

public String getOrchardName() { return orchardName; }
public String getFruitName() { return fruitName; }
public String getQuality() { return quality; }

public abstract String getDescription();

@Override
public boolean matchesFruit(String name) { return fruitName.equalsIgnoreCase(name); }
@Override
public boolean matchesQuality(String q) { return quality.equalsIgnoreCase(q); }
@Override
public String toString() {
return String.format("| %-10s | %-10s | %-8s | %-40s |",
orchardName, fruitName, quality, getDescription());
}
}
class OrchardA extends Orchard {
public OrchardA() { super("A", "Apple", "good"); }
@Override public String getDescription() { return "Crisp, fresh Apples - good quality"; }
}
class OrchardB extends Orchard {
public OrchardB() { super("B", "Litchi", "good"); }
@Override public String getDescription() { return "Juicy Litchis - good quality"; }
}
class OrchardC extends Orchard {
public OrchardC() { super("C", "Mango", "good"); }
@Override public String getDescription() { return "Sweet Mangoes - good quality"; }
}
class OrchardD extends Orchard {
public OrchardD() { super("D", "Apple", "bad"); }
@Override public String getDescription() { return "Apples - bad quality (over-ripe)"; }
}

class TreeNode {
private final String label;
private final Orchard orchard;
private final List<TreeNode> children;

public TreeNode(String label) {
this.label = label;
this.orchard = null;
this.children = new ArrayList<>();
}
```

```java
public TreeNode(Orchard orchard) {
this.label = "Orchard " + orchard.getOrchardName();
this.orchard = orchard;
this.children = new ArrayList<>();
}

public void addChild(TreeNode child) { children.add(child); }
public List<TreeNode> getChildren() { return children; }
public String getLabel() { return label; }
public Orchard getOrchard() { return orchard; }
public boolean isLeaf() { return orchard != null; }
public void printTree(String prefix, boolean isLast, String highlight) {

String connector = isLast ? "\\-- " : "+-- ";
String extension = isLast ? " " : "| ";
String display = label;
boolean matched = false;
if (isLeaf()) {
     String fruit = orchard.getFruitName();
     String quality = orchard.getQuality();
if (highlight != null && fruit.equalsIgnoreCase(highlight)) {
     display = label + " -> [* " + fruit.toUpperCase()
                        + " | " + quality + "] <-- MATCH";
                        matched = true;
} else {
     display = label + " -> " + fruit + " [" + quality + "]";
     }
}

if (highlight != null && isLeaf() && !matched) {
System.out.println(prefix + connector + display + " (no match)");
} else {
System.out.println(prefix + connector + display);
}

for (int i = 0; i < children.size(); i++) {
boolean childIsLast = (i == children.size() - 1);
children.get(i).printTree(prefix + extension, childIsLast, highlight);
            }
     }
}

public class Problem3_OrchardSystem {
static final String TABLE_HEADER =
"+------------+------------+----------+-----------------------------------------+\n"
+ "| Orchard | Fruit | Quality | Description |\n"
+ "+------------+------------+----------+-----------------------------------------+";
```

```java
static final String TABLE_FOOTER =
"+------------+------------+----------+----------------------------------------+";
static TreeNode buildTree(List<Orchard> orchards) {
TreeNode root = new TreeNode("Orchard System");
for (Orchard o : orchards) {
root.addChild(new TreeNode(o));
}
return root;
}
static void printTree(TreeNode root, String highlight) {
System.out.println(root.getLabel());
List<TreeNode> children = root.getChildren();
for (int i = 0; i < children.size(); i++) {
children.get(i).printTree("", i == children.size() - 1, highlight);
}
}
static void searchAndPrintTree(List<Orchard> orchards,
TreeNode root, String fruit) {
System.out.println("\n==============================================");
System.out.println(" Search Results -> Fruit: \"" + fruit + "\"");
System.out.println("==============================================");

boolean found = false;
for (Orchard o : orchards) {
if (o.matchesFruit(fruit)) {
if (!found) { System.out.println(TABLE_HEADER); found = true; }
System.out.println(o);
}
}
if (found) System.out.println(TABLE_FOOTER);
else System.out.println(" No orchard grows \"" + fruit + "\".");
System.out.println("\n Tree View (* = matched orchard):\n");
printTree(root, fruit);
}

public static void main(String[] args) {
List<Orchard> orchards = new ArrayList<>();
orchards.add(new OrchardA());
orchards.add(new OrchardB());
orchards.add(new OrchardC());
orchards.add(new OrchardD());
TreeNode root = buildTree(orchards);
System.out.println("╔═══════════════════════════════════════════════╗");
System.out.println("║ Orchard Management System (v2) ║");
System.out.println("╚═══════════════════════════════════════════════╝");
System.out.println("\n-- All Orchards (Tabular View) --");
System.out.println(TABLE_HEADER);
```

```java
for (Orchard o : orchards) System.out.println(o);
System.out.println(TABLE_FOOTER);
System.out.println("\n-- All Orchards (Tree View) --\n");
printTree(root, null);
Scanner sc = new Scanner(System.in);
System.out.print("\nEnter fruit name to search (Apple / Litchi / Mango): ");
String query = sc.nextLine().trim();
searchAndPrintTree(orchards, root, query);
sc.close();
     }
}
```

OUTPUT:

```
╔═══════════════════════════════════════════════╗
║         Orchard Management System  (v2)        ╕
╚═══════════════════════════════════════════════╝

-- All Orchards (Tabular View) --
+————————————+————————————+————————————+————————————————————————————————————+
| Orchard    | Fruit      | Quality    | Description                                |
+————————————+————————————+————————————+————————————————————————————————————+
| A          | Apple      | good       | Crisp, fresh Apples - good quality         |
| B          | Litchi     | good       | Juicy Litchis - good quality               |
| C          | Mango      | good       | Sweet Mangoes - good quality               |
| D          | Apple      | bad        | Apples - bad quality (over-ripe)           |
+————————————+————————————+————————————+————————————————————————————————————+

-- All Orchards (Tree View) --

Orchard System
+-- Orchard A  →  Apple [good]
+-- Orchard B  →  Litchi [good]
+-- Orchard C  →  Mango [good]
\-- Orchard D  →  Apple [bad]

Enter fruit name to search (Apple / Litchi / Mango): Apple

═══════════════════════════════════════════════
  Search Results  →  Fruit: "Apple"
═══════════════════════════════════════════════

+————————————+————————————+————————————+————————————————————————————————————+
| Orchard    | Fruit      | Quality    | Description                                |
+————————————+————————————+————————————+————————————————————————————————————+
| A          | Apple      | good       | Crisp, fresh Apples - good quality         |
```

```
| D          | Apple      | bad      | Apples - bad quality (over-ripe)         |
+------------+------------+----------+------------------------------------------+

  Tree View (* = matched orchard):

Orchard System
+-- Orchard A  ->  [* APPLE | good]  <-- MATCH
+-- Orchard B  ->  Litchi [good]  (no match)
+-- Orchard C  ->  Mango [good]  (no match)
\-- Orchard D  ->  [* APPLE | bad]  <-- MATCH

Enter fruit name to search (Apple / Litchi / Mango): Banana

================================================
  Search Results  ->  Fruit: "Banana"
================================================
  No orchard grows "Banana".

  Tree View (* = matched orchard):

Orchard System
+-- Orchard A  ->  Apple [good]  (no match)
+-- Orchard B  ->  Litchi [good]  (no match)
+-- Orchard C  ->  Mango [good]  (no match)
\-- Orchard D  ->  Apple [bad]  (no match)
```

4. Take a map provided as a graph & display all the subtrees of connected points.

```java
import java.util.*;


class Graph<T> {
private final Map<T, List<T>> adjList;
public Graph() {
this.adjList = new HashMap<>();
}
public void addNode(T node) {
adjList.putIfAbsent(node, new LinkedList<>());
}
public void addEdge(T u, T v) {
```

```
adjList.putIfAbsent(u, new LinkedList<>());
adjList.putIfAbsent(v, new LinkedList<>());
adjList.get(u).add(v);
adjList.get(v).add(u);
}
public Set<T> getNodes() {
return adjList.keySet();
}
public List<T> getNeighbors(T node) {
return adjList.getOrDefault(node, Collections.emptyList());
}
}
class SubtreeFinder<T> {
private final Graph<T> graph;
public SubtreeFinder(Graph<T> graph) {
this.graph = graph;
}
public List<List<T>> findAllSubtrees() {
Set<T> visited = new HashSet<>();
List<List<T>> allSubtrees = new ArrayList<>();
for (T node : graph.getNodes()) {
if (!visited.contains(node)) {
List<T> subtree = new ArrayList<>();
dfs(node, visited, subtree);
Collections.sort((List) subtree);
allSubtrees.add(subtree);
     }
}
return allSubtrees;
}
private void dfs(T current, Set<T> visited, List<T> subtree) {
visited.add(current);
subtree.add(current);
for (T neighbor : graph.getNeighbors(current)) {
if (!visited.contains(neighbor)) {
dfs(neighbor, visited, subtree);
                  }
            }
     }
}

class SubtreePrinter<T> {
public void print(List<List<T>> subtrees) {
System.out.println("\n╔═══════════════════════════════════════╗");
System.out.println("║        All Subtrees (Connected Groups) ║");
System.out.println("╚═══════════════════════════════════════╝");
int idx = 1;
```

```java
for (List<T> subtree : subtrees) {
System.out.println("\n Subtree #" + idx++ + ":");
System.out.print(" Nodes : ");
System.out.println(subtree);
System.out.print(" Visual: ");
for (int i = 0; i < subtree.size(); i++) {
System.out.print(subtree.get(i));
if (i < subtree.size() - 1) System.out.print(" — ");
}
System.out.println();
}
System.out.println("\n Total Subtrees Found: " + subtrees.size());
}
}
public class Problem4_AllSubtrees {
public static void main(String[] args) {
Graph<String> graph = new Graph<>();
graph.addEdge("A", "B");
graph.addEdge("B", "C");
graph.addEdge("B", "D");

graph.addEdge("E", "F");
graph.addEdge("F", "G");
graph.addNode("Z");
SubtreeFinder<String> finder = new SubtreeFinder<>(graph);
List<List<String>> subtrees = finder.findAllSubtrees();
SubtreePrinter<String> printer = new SubtreePrinter<>();
printer.print(subtrees);
}
}
```

OUTPUT:

```
╔═══════════════════════════════════╗
║     All Subtrees (Connected Groups)  ║
╚═══════════════════════════════════╝


 Subtree #1:
   Nodes : [A, B, C, D]
   Visual: A — B — C — D


 Subtree #2:
```

```
    Nodes : [E, F, G]
    Visual: E ── F ── G


  Subtree #3:
    Nodes : [Z]
    Visual: Z


Total Subtrees Found: 3
```

5. Enter entry point & goal point & produce the matching subtree of the graph.

```java
import java.util.*;


class GraphP5<T> {
private final Map<T, List<T>> adjList = new HashMap<>();
public void addEdge(T u, T v) {
adjList.putIfAbsent(u, new ArrayList<>());
adjList.putIfAbsent(v, new ArrayList<>());
adjList.get(u).add(v);
adjList.get(v).add(u);
}
public void addNode(T n) {
adjList.putIfAbsent(n, new ArrayList<>());
}

public Set<T> getNodes() { return adjList.keySet(); }
public List<T> getNeighbors(T node) {
return adjList.getOrDefault(node, Collections.emptyList());
}
public boolean hasNode(T node) { return adjList.containsKey(node); }
}
class PathResult<T> {
private final T start;
private final T goal;
private final List<T> path;
private final Set<T> subtreeNodes;
private final boolean found;

public PathResult(T start, T goal, List<T> path, Set<T> subtreeNodes, boolean found) {
this.start = start;
```

```
this.goal = goal;
this.path = Collections.unmodifiableList(new ArrayList<>(path));
this.subtreeNodes = Collections.unmodifiableSet(new LinkedHashSet<>(subtreeNodes));
this.found = found;
}
public T getStart() { return start; }
public T getGoal() { return goal; }
public List<T> getPath() { return path; }
public Set<T> getSubtreeNodes(){ return subtreeNodes; }
public boolean isFound() { return found; }
}
class PathFinder<T> {
private final GraphP5<T> graph;
public PathFinder(GraphP5<T> graph) {
this.graph = graph;
}

public PathResult<T> find(T start, T goal) {
if (!graph.hasNode(start) || !graph.hasNode(goal)) {
System.out.println(" [Error] Node not found in graph.");
return new PathResult<>(start, goal, List.of(), Set.of(), false);
}
Queue<T> queue = new LinkedList<>();
Map<T, T> parent = new HashMap<>();
Set<T> visited = new HashSet<>();
queue.add(start);
visited.add(start);
parent.put(start, null);
boolean reached = false;

while (!queue.isEmpty()) {
T current = queue.poll();
if (current.equals(goal)) {
reached = true;
break;
}
for (T neighbor : graph.getNeighbors(current)) {
if (!visited.contains(neighbor)) {
visited.add(neighbor);
parent.put(neighbor, current);
queue.add(neighbor);
}
}
}

if (!reached) {
return new PathResult<>(start, goal, List.of(), visited, false);
```

```
}

List<T> path = new ArrayList<>();
for (T cur = goal; cur ≠ null; cur = parent.get(cur)) {
path.add(cur);
}
Collections.reverse(path);

Set<T> subtree = collectSubtree(start);
return new PathResult<>(start, goal, path, subtree, true);
}

private Set<T> collectSubtree(T source) {
Set<T> visited = new LinkedHashSet<>();
Deque<T> stack = new ArrayDeque<>();
stack.push(source);
while (!stack.isEmpty()) {
T node = stack.pop();
if (visited.add(node)) {
for (T neighbor : graph.getNeighbors(node)) {
if (!visited.contains(neighbor)) stack.push(neighbor);
}
}
}
return visited;
}
}
class PathResultPrinter<T> {
public void print(PathResult<T> result) {
System.out.println("\n╔═══════════════════════════════════════╗");
System.out.println("║ Problem 5 — Entry → Goal Subtree ║");
System.out.println("╚═══════════════════════════════════════╝");
System.out.println(" Entry Point : " + result.getStart());
System.out.println(" Goal Point : " + result.getGoal());

if (!result.isFound()) {
System.out.println("\n ✗ No path exists between these two nodes.");
System.out.println(" (They may belong to different connected components)");
return;
}

System.out.println("\n ✓ Path Found (BFS shortest hop count):");
System.out.print(" ");
List<T> path = result.getPath();
for (int i = 0; i < path.size(); i++) {
System.out.print(path.get(i));
if (i < path.size() - 1) System.out.print(" → ");
```

```java
}
System.out.println();
System.out.println(" Hops: " + (path.size() - 1));

System.out.println("\n Matching Subtree (connected component):");
System.out.println(" Nodes: " + result.getSubtreeNodes());
}
}

public class Problem5_EntryGoalSubtree {
public static void main(String[] args) {
GraphP5<String> graph = new GraphP5<>();
graph.addEdge("A", "B");
graph.addEdge("A", "C");
graph.addEdge("B", "D");
graph.addEdge("B", "E");
graph.addEdge("D", "F");
graph.addEdge("F", "G");
graph.addEdge("X", "Y");
PathFinder<String> finder = new PathFinder<>(graph);
PathResultPrinter<String> printer = new PathResultPrinter<>();
System.out.println("\n──────────────── Test 1 ────────────────");
PathResult<String> r1 = finder.find("A", "G");
printer.print(r1);
System.out.println("\n──────────────── Test 2 ────────────────");
PathResult<String> r2 = finder.find("C", "E");
printer.print(r2);
System.out.println("\n──────────────── Test 3 ────────────────");
PathResult<String> r3 = finder.find("A", "X");
printer.print(r3);
}
}
```

OUTPUT:

```
────────────────  Test 1  ────────────────

╔═══════════════════════════════════════╗
║   Problem 5 ── Entry → Goal Subtree     ║
╚═══════════════════════════════════════╝

 Entry Point : A

 Goal  Point : G

 ✔ Path Found (BFS shortest hop count):
```

```
    A → B → D → F → G
    Hops: 4
  Matching Subtree (connected component):
    Nodes: [A, C, B, E, D, F, G]
──────────────── Test 2 ────────────────

╔═══════════════════════════════════════╗
║   Problem 5 — Entry → Goal Subtree     ║
╚═══════════════════════════════════════╝

  Entry Point : C
  Goal  Point : E
  ✔ Path Found (BFS shortest hop count):
    C → A → B → E
    Hops: 3
  Matching Subtree (connected component):
    Nodes: [C, A, B, E, D, F, G]
──────────────── Test 3 ────────────────

╔═══════════════════════════════════════╗
║   Problem 5 — Entry → Goal Subtree     ║
╚═══════════════════════════════════════╝

  Entry Point : A
  Goal  Point : X


  ✘ No path exists between these two nodes.
(They may belong to different connected components)
```

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
import java.util.*;

class WeightedGraph<T> {
static class Edge<T> {
final T destination;
final int weight;

Edge(T destination, int weight) {
this.destination = destination;
this.weight = weight;
}
}
private final Map<T, List<Edge<T>>> adjList = new HashMap<>();
public void addEdge(T u, T v, int weight) {
adjList.putIfAbsent(u, new ArrayList<>());
adjList.putIfAbsent(v, new ArrayList<>());
adjList.get(u).add(new Edge<>(v, weight));
adjList.get(v).add(new Edge<>(u, weight));
}
public void addNode(T n) {
adjList.putIfAbsent(n, new ArrayList<>());
}

public Set<T> getNodes() { return adjList.keySet(); }
public List<Edge<T>> getEdges(T node) {
return adjList.getOrDefault(node, Collections.emptyList());
}
public boolean hasNode(T n) { return adjList.containsKey(n); }
}
class NodeEntry<T> implements Comparable<NodeEntry<T>> {
final T node;
final int dist;
NodeEntry(T node, int dist) {
this.node = node;
this.dist = dist;
}

@Override
public int compareTo(NodeEntry<T> other) {
return Integer.compare(this.dist, other.dist);
}
}
class ShortestPathResult<T> {
private final T start;
```

```java
private final T end;
private final List<T> path;
private final int totalCost;
private final boolean reachable;

ShortestPathResult(T start, T end, List<T> path, int totalCost, boolean reachable) {
this.start = start;
this.end = end;
this.path = Collections.unmodifiableList(new ArrayList<>(path));
this.totalCost = totalCost;
this.reachable = reachable;
}

public T getStart() { return start; }
public T getEnd() { return end; }
public List<T> getPath() { return path; }
public int getTotalCost() { return totalCost; }
public boolean isReachable(){ return reachable; }
}
class DijkstraSolver<T> {
private final WeightedGraph<T> graph;
DijkstraSolver(WeightedGraph<T> graph) {
this.graph = graph;
}

public ShortestPathResult<T> shortestPath(T start, T end) {
if (!graph.hasNode(start) || !graph.hasNode(end)) {
return new ShortestPathResult<>(start, end, List.of(), -1, false);
}

Map<T, Integer> dist = new HashMap<>();
Map<T, T> parent = new HashMap<>();

for (T node : graph.getNodes()) dist.put(node, Integer.MAX_VALUE);
dist.put(start, 0);


PriorityQueue<NodeEntry<T>> pq = new PriorityQueue<>();
pq.add(new NodeEntry<>(start, 0));

Set<T> settled = new HashSet<>();

while (!pq.isEmpty()) {
NodeEntry<T> entry = pq.poll();
T current = entry.node;

if (settled.contains(current)) continue;
```

```
settled.add(current);

if (current.equals(end)) break;

for (WeightedGraph.Edge<T> edge : graph.getEdges(current)) {
T neighbor = edge.destination;
if (settled.contains(neighbor)) continue;

int newDist = dist.get(current) + edge.weight;
if (newDist < dist.get(neighbor)) {
dist.put(neighbor, newDist);
parent.put(neighbor, current);
pq.add(new NodeEntry<>(neighbor, newDist));
}
}
}
if (dist.get(end) == Integer.MAX_VALUE) {
return new ShortestPathResult<>(start, end, List.of(), -1, false);
}
list<T> path = new ArrayList<>();
for (T cur = end; cur ≠ null; cur = parent.get(cur)) {
path.add(cur);
}
Collections.reverse(path);

return new ShortestPathResult<>(start, end, path, dist.get(end), true);
}
}
class ShortestPathPrinter<T> {
public void print(ShortestPathResult<T> result) {
System.out.println("\n╔═══════════════════════════════════════╗");
System.out.println("║ Problem 6 — Dijkstra Shortest Path          ║");
System.out.println("╚═══════════════════════════════════════╝");
System.out.println(" Start : " + result.getStart());
System.out.println(" End : " + result.getEnd());

if (!result.isReachable()) {
System.out.println("\n ✗ No path found (nodes may be disconnected).");
return;
}
System.out.println("\n ✔ Shortest Path:");
System.out.print(" Route : ");
List<T> path = result.getPath();
for (int i = 0; i < path.size(); i++) {
System.out.print(path.get(i));
if (i < path.size() - 1) System.out.print(" ──► ");
}
```

```java
System.out.println();
System.out.println(" Total Cost (weight) : " + result.getTotalCost());
System.out.println(" Nodes visited : " + path.size());
}
}
public class Problem6_ShortestPath {

public static void main(String[] args) {
WeightedGraph<String> graph = new WeightedGraph<>();
graph.addEdge("A", "B", 4);
graph.addEdge("A", "C", 1);
graph.addEdge("B", "D", 3);
graph.addEdge("B", "E", 2);
graph.addEdge("C", "D", 5);
graph.addEdge("D", "F", 6);
graph.addEdge("E", "F", 1);
graph.addEdge("G", "H", 10);
DijkstraSolver<String> solver = new DijkstraSolver<>(graph);
ShortestPathPrinter<String> printer = new ShortestPathPrinter<>();
System.out.println("\n──────────── Test 1: A → F ────────────");
printer.print(solver.shortestPath("A", "F"));
System.out.println("\n──────────── Test 2: C → E ────────────");
printer.print(solver.shortestPath("C", "E"));
System.out.println("\n──────────── Test 3: A → H ────────────");
printer.print(solver.shortestPath("A", "H"));
System.out.println("\n──────────── Test 4: A → A ────────────");
printer.print(solver.shortestPath("A", "A"));
}
}
```

OUTPUT:

```
──────────── Test 1: A → F ────────────

╔═══════════════════════════════════════╗
║                                       ╕
│   Problem 6 — Dijkstra Shortest Path  │
╘                                       ╛
╚═══════════════════════════════════════╝

 Start : A

 End   : F

 ✔ Shortest Path:

   Route : A  ──►  B  ──►  E  ──►  F

   Total Cost (weight) : 7
```

```
    Nodes visited       : 4
──────────── Test 2: C → E ────────────

╔═══════════════════════════════════════╗
║   Problem 6 — Dijkstra Shortest Path  ║
╚═══════════════════════════════════════╝

  Start : C
  End   : E
  ✔ Shortest Path:
    Route : C  ──►  A  ──►  B  ──►  E
    Total Cost (weight) : 7
    Nodes visited       : 4
──────────── Test 3: A → H ────────────

╔═══════════════════════════════════════╗
║   Problem 6 — Dijkstra Shortest Path  ║
╚═══════════════════════════════════════╝

  Start : A
  End   : H
  ✘ No path found (nodes may be disconnected).
──────────── Test 4: A → A ────────────

╔═══════════════════════════════════════╗
║   Problem 6 — Dijkstra Shortest Path  ║
╚═══════════════════════════════════════╝

  Start : A
  End   : A
  ✔ Shortest Path:
    Route : A
    Total Cost (weight) : 0
Nodes visited       : 1
```

7.Write a program to demonstrate the use of all keywords made for exception handling.

```
class InsufficientAgeException extends Exception {
public InsufficientAgeException(String message) {
super(message);
}
}

public class ExceptionKeywordsDemo {
static void validateAge(int age) throws InsufficientAgeException {
if (age < 18) {
throw new InsufficientAgeException("Age " + age + " is below 18.");
}
System.out.println("Age " + age + " is valid.");
}

public static void main(String[] args) {

try {
validateAge(25);
validateAge(15);

} catch (InsufficientAgeException e) {
System.out.println("Custom Exception caught: " + e.getMessage());

} catch (Exception e) {
System.out.println("General Exception caught: " + e.getMessage());

} finally {
System.out.println("finally block: This always runs.");
}

System.out.println("Program continues after exception handling.");
}}
```

OUTPUT:

```
Age 25 is valid.

Custom Exception caught: Age 15 is below 18.

finally block: This always runs.

Program continues after exception handling.
```

8. Write a program to with raw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

```
class NoMoneyException extends RuntimeException {
private double balance;
private double withdrawalAmount;

public NoMoneyException(double balance, double withdrawalAmount) {
super(String.format(
"Withdrawal of %.2f failed. Available balance: %.2f",
withdrawalAmount, balance
));
this.balance = balance;
this.withdrawalAmount = withdrawalAmount;
}

public double getBalance() { return balance; }
public double getWithdrawalAmount() { return withdrawalAmount; }
}
class BankAccount {
private String owner;
private double balance;

public BankAccount(String owner, double initialBalance) {
this.owner = owner;
this.balance = initialBalance;
}
public void withdraw(double amount) {
if (amount ≤ 0) {
throw new IllegalArgumentException("Withdrawal amount must be positive.");
}
if (amount > balance) {

throw new NoMoneyException(balance, amount);
}
balance -= amount;
System.out.printf("Withdrawn: %.2f | Remaining balance: %.2f%n", amount, balance);
}

public double getBalance() { return balance; }
public String getOwner() { return owner; }
}

public class BankDemo {
public static void main(String[] args) {
```

```java
BankAccount account = new BankAccount("Alice", 5000.00);
System.out.println("Account owner: " + account.getOwner());

try {
account.withdraw(1500.00);
account.withdraw(4000.00);
} catch (NoMoneyException e) {
System.out.println("ERROR: " + e.getMessage());
System.out.printf("You tried to withdraw %.2f but only %.2f is available.%n",
e.getWithdrawalAmount(), e.getBalance());
} catch (IllegalArgumentException e) {
System.out.println("Invalid input: " + e.getMessage());
} finally {
System.out.printf("Final balance for %s: %.2f%n",
account.getOwner(), account.getBalance());
}
}
}
```

OUTPUT:

```
Account owner: Rajiv

Withdrawn: 1500.00 | Remaining balance: 3500.00

ERROR: Withdrawal of 4000.00 failed. Available balance: 3500.00

You tried to withdraw 4000.00 but only 3500.00 is available.

Final balance for Rajiv: 3500.00
```

9. Write a program which enables to use a platform by a train one at a time.

```
class Platform {
private String platformName;
private boolean isOccupied = false;

public Platform(String name) {
this.platformName = name;
}
public synchronized void usePlatform(String trainName) {
while (isOccupied) {
System.out.println(trainName + " is waiting — platform is busy.");
try {
wait();
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}
isOccupied = true;
System.out.println(">>> " + trainName + " has entered " + platformName);
try {
Thread.sleep(1500);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
System.out.println("<<< " + trainName + " has left " + platformName);
isOccupied = false;
notifyAll();
}
}

class Train implements Runnable {
private String name;
private Platform platform;

public Train(String name, Platform platform) {
this.name = name;
this.platform = platform;
}

@Override
public void run() {
platform.usePlatform(name);
}
}

public class RailwayPlatformDemo {
```

```java
public static void main(String[] args) throws InterruptedException {
Platform platform = new Platform("Platform 1");
Thread t1 = new Thread(new Train("Express 101", platform));
Thread t2 = new Thread(new Train("Local 202", platform));
Thread t3 = new Thread(new Train("Freight 303", platform));
Thread t4 = new Thread(new Train("Metro 404", platform));
t1.start();
t2.start();
t3.start();
t4.start();
t1.join();
t2.join();
t3.join();
t4.join();
System.out.println("All trains have used the platform.");
}
}
```

OUTPUT:

```
>>> Express 101 has entered Platform 1

Local 202 is waiting — platform is busy.

Freight 303 is waiting — platform is busy.

<<< Express 101 has left Platform 1

>>> Local 202 has entered Platform 1

Freight 303 is waiting — platform is busy.

...

All trains have used the platform.
```

10. Write a program of Restaurant and Waiter, which accept order if kitchen if free and chef deliver orders if he is free.

```
import java.util.LinkedList;

import java.util.Queue;

class Kitchen {
private final Queue<String> orderQueue = new LinkedList<>();
private final int MAX_ORDERS = 3;
private boolean isOpen = true;


public synchronized void placeOrder(String dish) throws InterruptedException {
while (orderQueue.size() ≥ MAX_ORDERS) {
System.out.println("[Waiter] Kitchen full, waiting to place: " + dish);
wait();
}
orderQueue.add(dish);
System.out.println("[Waiter] Order placed: " + dish
+ " | Queue size: " + orderQueue.size());
notifyAll();
}

public synchronized String deliverOrder() throws InterruptedException {
while (orderQueue.isEmpty() && isOpen) {
System.out.println("[Chef] No orders yet, waiting...");
wait();
}
if (orderQueue.isEmpty()) return null;

String dish = orderQueue.poll();
System.out.println("[Chef] Preparing: " + dish
+ " | Remaining: " + orderQueue.size());
notifyAll();
return dish;
}
public synchronized void closeKitchen() {
isOpen = false;
notifyAll();
}

public boolean isOpen() { return isOpen; }
}

class Waiter implements Runnable {
```

```java
private Kitchen kitchen;
private String[] orders = {
"Pasta", "Pizza", "Burger", "Salad", "Soup", "Steak"
};

public Waiter(Kitchen kitchen) {
this.kitchen = kitchen;
}

@Override
public void run() {
try {
for (String order : orders) {
kitchen.placeOrder(order);
Thread.sleep(400);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
} finally {
kitchen.closeKitchen();
System.out.println("[Waiter] All orders placed. Closing kitchen.");
}
}
}
class Chef implements Runnable {
private Kitchen kitchen;

public Chef(Kitchen kitchen) {
this.kitchen = kitchen;
}

@Override
public void run() {
try {
while (kitchen.isOpen() || true) {
String dish = kitchen.deliverOrder();
if (dish == null) break;
System.out.println("[Chef] Served: " + dish);
Thread.sleep(700);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
System.out.println("[Chef] Done for the day.");
}
}
```

```java
public class RestaurantDemo {
public static void main(String[] args) throws InterruptedException {
Kitchen kitchen = new Kitchen();
Thread waiterThread = new Thread(new Waiter(kitchen), "Waiter-Thread");
Thread chefThread = new Thread(new Chef(kitchen), "Chef-Thread");
chefThread.start();
waiterThread.start();
waiterThread.join();
chefThread.join();

System.out.println("Restaurant closed for the day.");
}
}
```

OUTPUT:

```
[Chef] No orders yet, waiting...
[Waiter] Order placed: Pasta     | Queue size: 1
[Chef] Preparing: Pasta          | Remaining: 0
[Waiter] Order placed: Pizza     | Queue size: 1
[Waiter] Order placed: Burger    | Queue size: 2
[Chef] Served: Pasta
[Chef] Preparing: Pizza          | Remaining: 1
[Waiter] Order placed: Salad     | Queue size: 2
...
Restaurant closed for the day.
```